

Contexto

O Hospital “São Binário” precisa modernizar o sistema de triagem do Pronto-Socorro. Atualmente, os pacientes entram em uma fila simples (primeiro a chegar, primeiro a ser atendido). O novo sistema deve atender os pacientes em ordem de **prioridade clínica**, não de chegada. Sua tarefa é implementar esse sistema usando uma Fila com Prioridade baseada em **Heap Binário**.

A entidade Paciente

Cada paciente possui três atributos que, **em conjunto**, determinam sua prioridade de atendimento:

Atributo	Tipo	Descrição
nome	String	Nome do paciente
nivelUrgencia	int (1 a 5)	Protocolo simplificado (veja tabela abaixo)
tempoEsperaMinutos	int	Minutos que o paciente aguarda atendimento
grupoVulneravel	boolean	true se o paciente tem menos de 12 ou mais de 65 anos

Níveis de urgência

Nível	Cor	Classificação
5	Vermelho	Imediato
4	Laranja	Muito Urgente
3	Amarelo	Urgente
2	Verde	Pouco Urgente
1	Azul	Não Urgente

Regras de Prioridade

A prioridade entre dois pacientes é determinada por **três critérios em cascata** (o próximo só é consultado em caso de empate no anterior):

1. **Nível de urgência:** paciente com `nivelUrgencia` maior é atendido primeiro.
2. **Tempo de espera:** para o mesmo nível, paciente com maior `tempoEsperaMinutos` é atendido primeiro.
3. **Grupo vulnerável:** para mesmo nível e tempo, paciente com `grupoVulneravel == true` é atendido primeiro.

Como testar

A aplicação deve registrar a chegada dos **6 pacientes** abaixo e imprimir o estado interno do heap após cada inserção. Em seguida, remover todos os pacientes em ordem de prioridade.

Pacientes de teste

#	Nome	nivelUrgencia	tempoEspera (min)	grupoVulneravel
1	Carlos	2	45	false
2	Maria	5	5	false
3	João	3	20	false
4	Beatriz	3	35	true
5	Pedro	5	2	false
6	Helena	2	45	true

Ordem de atendimento esperada:

1. Maria: Vermelho, maior espera entre os críticos
2. Pedro: Vermelho, mesma urgência, menor espera
3. Beatriz: Amarelo, maior espera e vulnerável
4. João: Amarelo, menor espera
5. Helena: Verde, mesma espera de Carlos, mas vulnerável
6. Carlos: Verde, última prioridade

Se a saída da aplicação estiver de acordo com essa ordem, a implementação está correta.

Requisitos de Implementação

Sua aplicação deve seguir os requisitos abaixo:

1. Utilização de **Heap** para implementação da Fila de Prioridade;
2. Criar um fork do projeto [07-priority-queue-heap](#);
3. Implementar o método de **enfileirar** com o algoritmo “**Sobe Heap**”.
4. Implementar o método de **desenfileirar** com o algoritmo “**Desce Heap**”.
5. Implementar a entidade **Paciente** com as regras da seção anterior.
6. **README.md** elaborado seguindo as boas práticas apresentadas durante a disciplina (descrição, requisitos, compilação, execução, etc).
7. Publicação do **Javadoc** do projeto no **Github Pages**.

Setup Github Pages (Javadoc):

O arquivo “javadoc.yaml” da pasta “.github/workflows” do projeto já realiza o processo de publicação do projeto no Github Pages automaticamente. Entretanto, a configuração abaixo é necessária para que o deploy seja feito corretamente.

Habilitar o deploy da página no GH Pages:

Acessar: Settings → Pages → Source → selecionar "GitHub Actions"

